

Benchmarking Claude Code: Anthropic vs DeepSeek Model Backends

An applied research assignment in AI agent evaluation

1. Context and the question you're answering

Claude Code is a coding agent, not a language model. It ships with a fixed "agent loop" — the harness that decides when to read files, run shell commands, edit code, and re-plan — and it calls out to a language model on every turn to do the actual thinking. The default backend is Anthropic's Claude models, but Claude Code is designed to be model-agnostic: any provider that speaks the Anthropic API format can be plugged in by setting two environment variables.

This raises a research question with real economic stakes. DeepSeek V4-Pro is roughly an order of magnitude cheaper per token than Claude Opus 4.7 and is reported by DeepSeek to score 80.6% on SWE-bench Verified — within striking distance of frontier proprietary models. If that quality claim holds up *inside the Claude Code harness*, on tasks the model has not been heavily optimized for, then teams shipping AI-driven software could meaningfully cut spend with no quality regression. The speed picture is less obvious: DeepSeek is cheaper but the inference may be slower per token, so end-to-end wall-clock time depends on how the harness uses the model.

The hypothesis under test:

When Claude Code is run with a DeepSeek model backend in place of Claude, **task success rate (quality) does not meaningfully diminish**, while **end-to-end wall-clock time per task drops materially** because DeepSeek's pricing economics and infrastructure favor higher throughput. Cost per resolved task drops by an even larger factor than time.

Your job is to design, run, and write up a study that either supports or falsifies this hypothesis with reproducible evidence.

2. What you're building

A public Git repository — yours, not a fork — containing:

1. A working setup that runs Claude Code against an industry-standard agent benchmark with three model backends (Anthropic, DeepSeek V4-Pro, DeepSeek V4-Flash).
2. Raw results from your benchmark runs (logs, JSON outputs, trial-level data).
3. An analysis notebook or document with quality, speed, and spend comparisons.

4. A README that lets a stranger reproduce your numbers in under an hour, plus a short write-up of your findings.

Share the repo with me (GitHub handle to follow) when complete. Read the rest of this document end to end before you start running anything.

3. Why Terminal-Bench 2.0 (and not SWE-bench)

The benchmark you'll use is **Terminal-Bench 2.0**, built by the Laude Institute and run via the open-source **Harbor** framework.

You will encounter SWE-bench Verified more often in the literature, and it's a defensible choice, but it has a specific limitation for our purposes: SWE-bench was designed to evaluate *language models* via a fixed scaffold (SWE-Agent). When Anthropic and others report Claude scores on SWE-bench, they're running Claude-the-model inside SWE-Agent-the-harness, not Claude Code. That means SWE-bench results don't isolate the question you're trying to answer, which is "how does the Claude Code agent — its tools, prompting, planning logic — perform with a different language model under the hood?"

Terminal-Bench 2.0 was designed for exactly this. It has 89 tasks spanning system administration, model training, debugging, security, and software engineering, each shipped as a Docker container with a deterministic test script. Harbor — the harness — has **Claude Code as a first-class supported agent**, invoked as `--agent claude-code`. Anthropic reported Claude Opus 4.6's headline agent score on Terminal-Bench 2.0, which means your results sit alongside numbers the labs themselves publish.

If you have time and budget left over, running a subset of SWE-bench Verified as a second benchmark is excellent extra credit (see §10).

4. Why the model swap works at all

Claude Code reads its backend configuration from environment variables. By default it points at Anthropic's API, but two variables override that:

```
export ANTHROPIC_BASE_URL="<provider's Anthropic-compatible endpoint>"
export ANTHROPIC_AUTH_TOKEN="<provider's API key>"
export ANTHROPIC_MODEL="<provider's model name>"
```

DeepSeek explicitly publishes an Anthropic-compatible endpoint at <https://api.deepseek.com/anthropic>. When Claude Code makes a `/v1/messages` call, DeepSeek receives it in Anthropic format, translates it internally, and responds in Anthropic format. The agent loop never knows it isn't talking to Claude. This is the same pattern Cloudflare AI Gateway, OpenRouter, and z.ai use to make non-Anthropic models work inside Claude Code.

The implication for your study: you are holding the agent harness, prompting, tool definitions, and task descriptions constant across all runs. The only thing that varies is the model on the other end of the API call. That is exactly the controlled experiment you want.

5. Environment setup

You need a Linux or macOS machine (Windows users: use WSL2). Recommended: 16GB RAM, 8 cores, 80GB+ free disk for Docker images. The full benchmark run takes a few hours of compute; the analysis is fast.

5.1 Prerequisites

```
# Node.js 22 (Claude Code requires it)
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.2/install.sh | bash
source ~/.nvm/nvm.sh
nvm install 22

# Python 3.11+ and uv (Harbor uses uv)
curl -LsSf https://astral.sh/uv/install.sh | sh

# Docker (running, with at least 60GB disk allocated)
# Install per https://docs.docker.com/engine/install/

# Claude Code
npm install -g @anthropic-ai/claude-code
claude --version # confirm it installed
```

5.2 Harbor + Terminal-Bench

```
# Create your project dir
mkdir cc-deepseek-bench && cd cc-deepseek-bench
git init

# Install Harbor (uses uv)
uv init
uv add harbor

# Verify Harbor can see Terminal-Bench 2.0
uv run harbor list-datasets | grep terminal-bench
```

5.3 API keys

You need accounts and API keys for two providers:

- **Anthropic** — sign up at console.anthropic.com, create an API key. Budget ~\$40–80 for the full study with Opus 4.7 as the Anthropic arm. (You can use Sonnet 4.6 instead to cut cost by ~5x; see §7.2.)
- **DeepSeek** — sign up at platform.deepseek.com, create an API key. They give 5M free tokens on signup. V4-Pro is currently at 75% off through May 31, 2026, which makes this experiment unusually cheap to run if you start soon.

Put both keys in a `.env` file at the repo root. **Do not commit it.** Add `.env` to `.gitignore` immediately:

```
# .env
ANTHROPIC_API_KEY=sk-ant-...
DEEPSEEK_API_KEY=sk-...
```

5.4 Sanity check

Before running the benchmark, prove the model swap works end to end with the hello-world task:

```
# Arm A: Claude Code with Anthropic backend
export ANTHROPIC_API_KEY=$(grep ANTHROPIC_API_KEY .env | cut -d= -f2)
unset ANTHROPIC_BASE_URL ANTHROPIC_AUTH_TOKEN ANTHROPIC_MODEL # clear any DeepSeek overrides

uv run harbor run \
  --dataset terminal-bench@2.0 \
  --agent claude-code \
  --model anthropic/claude-sonnet-4-6 \
  --task-id hello-world \
  --n-concurrent 1

# Arm B: Claude Code with DeepSeek backend
export ANTHROPIC_BASE_URL="https://api.deepseek.com/anthropic"
export ANTHROPIC_AUTH_TOKEN=$(grep DEEPSEEK_API_KEY .env | cut -d= -f2)
export ANTHROPIC_MODEL="deepseek-v4-pro"
export ANTHROPIC_DEFAULT_OPUS_MODEL="deepseek-v4-pro"
export ANTHROPIC_DEFAULT_SONNET_MODEL="deepseek-v4-pro"
export ANTHROPIC_DEFAULT_HAIKU_MODEL="deepseek-v4-flash"
export CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC=1

uv run harbor run \
  --dataset terminal-bench@2.0 \
  --agent claude-code \
  --model anthropic/claude-sonnet-4-6 \
  --task-id hello-world \
  --n-concurrent 1
```

Both should pass. If Arm B fails with an auth error, the env vars didn't propagate into the Harbor subprocess — check that `claude` itself (run interactively in the same shell) is routing to DeepSeek before continuing. A quick way to verify: `claude -p "what model are you?"` should mention DeepSeek when Arm B's env is active.

If you can't get both arms to pass `hello-world`, stop and debug. Do not proceed to the full benchmark until the sanity check is green for every arm you plan to run.

6. Experimental design

6.1 Arms

Run three configurations:

Arm	Backend	Model	Anthropic-compatible endpoint
A	Anthropic	claude-sonnet-4-6 (or claude-opus-4-7 if budget allows)	default
B	DeepSeek	deepseek-v4-pro	https://api.deepseek.com/anthropic
C	DeepSeek	deepseek-v4-flash	https://api.deepseek.com/anthropic

The three arms test different parts of the hypothesis. **A vs B** tests "can a non-Anthropic model match quality" — DeepSeek V4-Pro is positioned as a quality-competitive reasoning model. **A vs C** tests "what does the speed/cost play look like with the cheap variant" — V4-Flash is the workhorse model where the cost story is most dramatic.

6.2 Task subset

Running all 89 Terminal-Bench 2.0 tasks at 3 trials × 3 arms = 801 runs is more than you need and probably outside your budget. Run a **representative subset of 20 tasks** chosen as follows:

- Stratify by Terminal-Bench's difficulty tags: pick a mix of easy/medium/hard roughly matching the dataset distribution.
- Stratify by category: include tasks from at least 5 different categories (e.g. software-engineering, system-admin, data-science, security, model-training).
- Pick the same 20 task IDs for all three arms. Hardcode them in your repo so the subset is reproducible.

You can list available tasks and their metadata with `uv run harbor list-tasks --dataset terminal-bench@2.0`.

6.3 Trials

Run each (arm, task) pair **3 times**. Coding agents are non-deterministic — same prompt, same model, different result — and a single trial tells you almost nothing about whether a difference is real. Three trials lets you compute mean and a rough standard error per cell, and lets you report quality as $\text{successes} / (3 \times 20) = x / 60$ rather than a noisy single-shot percentage.

This is the single most important methodological point in the assignment. Do not skip it. If you have to choose between fewer tasks with three trials or more tasks with one trial, **always choose more trials**.

6.4 Metrics

For each individual run, capture:

- **Success** — did the task's pytest verifier pass? (binary, from Harbor's reward output)
- **Wall-clock duration** — seconds from agent start to verdict
- **Total input tokens** and **total output tokens** consumed
- **Cost** in USD, computed from token counts × provider's published per-token rate

- **Number of agent turns / tool calls** — useful for understanding *why* speed differs
- **Failure mode** (if failed) — categorize manually after the fact: refused-to-try, looped, ran-out-of-budget, produced-wrong-output, timed-out

Harbor writes a structured results JSON per run that contains most of this. The full Claude Code session transcript is also saved; you'll want it for the qualitative analysis.

6.5 Running the full sweep

```
# Anthropic arm — 20 tasks × 3 trials
unset ANTHROPIC_BASE_URL ANTHROPIC_AUTH_TOKEN ANTHROPIC_MODEL
export ANTHROPIC_API_KEY=$(grep ANTHROPIC_API_KEY .env | cut -d= -f2)
uv run harbor run \
  --dataset terminal-bench@2.0 \
  --agent claude-code \
  --model anthropic/claude-sonnet-4-6 \
  --task-ids-file ./tasks.txt \
  --n-attempts 3 \
  --n-concurrent 4 \
  --output-dir ./results/arm-a-anthropic

# DeepSeek V4-Pro arm
export ANTHROPIC_BASE_URL="https://api.deepseek.com/anthropic"
export ANTHROPIC_AUTH_TOKEN=$(grep DEEPSEEK_API_KEY .env | cut -d= -f2)
export ANTHROPIC_MODEL="deepseek-v4-pro"
export ANTHROPIC_DEFAULT_SONNET_MODEL="deepseek-v4-pro"
export CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC=1
uv run harbor run \
  --dataset terminal-bench@2.0 \
  --agent claude-code \
  --model anthropic/claude-sonnet-4-6 \
  --task-ids-file ./tasks.txt \
  --n-attempts 3 \
  --n-concurrent 4 \
  --output-dir ./results/arm-b-deepseek-pro

# DeepSeek V4-Flash arm
export ANTHROPIC_MODEL="deepseek-v4-flash"
export ANTHROPIC_DEFAULT_SONNET_MODEL="deepseek-v4-flash"
uv run harbor run \
  --dataset terminal-bench@2.0 \
  --agent claude-code \
  --model anthropic/claude-sonnet-4-6 \
  --task-ids-file ./tasks.txt \
  --n-attempts 3 \
  --n-concurrent 4 \
  --output-dir ./results/arm-c-deepseek-flash
```

Note that `--model` in the Harbor command is the *display name* for the agent — when Claude Code is the agent and the backend is overridden via env vars, the model that actually gets called is whatever `ANTHROPIC_MODEL` points at. Capture this carefully in your README; it's the most confusing part of the setup.

Set `--n-concurrent` based on what your machine can handle. Start at 4 and watch CPU/memory.

7. Analysis you need to produce

Put this in a Jupyter notebook `analysis.ipynb` or a markdown report `analysis.md` in the repo. It must answer four questions.

7.1 Quality

Compute, per arm:

- Overall success rate across all 60 (task, trial) cells, with a 95% confidence interval (use Wilson score interval — Wald is wrong at small n).
- Per-task success rate (0, 1/3, 2/3, or 3/3) as a table.
- Tasks where the arms diverge — i.e., one arm solved it consistently and another didn't. These are the most interesting tasks. Pull the transcripts and read them.

The headline number is "quality didn't drop" only if the confidence intervals overlap meaningfully. If DeepSeek's CI sits clearly below Anthropic's, the hypothesis is falsified.

7.2 Speed

Per arm, report:

- Mean and median wall-clock seconds per task.
- Distribution across tasks (a box plot is the right chart here).
- Speed broken down by success vs failure — failed runs often hit a time limit, which can mask real differences.

Then derive: per-task, compute the speedup ratio $T_{anthropic} / T_{deepseek}$ and report the median across tasks. A median speedup of 1.5x is a real result; 1.05x is noise.

7.3 Cost

Compute total spend per arm. Use these reference prices (verify against the provider's page on the day you run — these move):

Model	Input \$/M	Output \$/M
c laude-sonnet-4-6	\$3.00	\$15.00
c laude-opus-4-7	\$15.00	\$75.00
deepseek-v4-pro	\$0.435	\$0.87 (during 75% discount; otherwise \$1.74 / \$3.48)
deepseek-v4-flash	\$0.14	\$0.28

The headline metric is **cost per resolved task** — total arm spend divided by tasks solved at least once. This normalizes for the fact that a cheaper-but-worse model might cost more per *successful* output even if its per-token rate is lower.

7.4 Qualitative

Pick 3–5 tasks where the arms diverged in interesting ways and read the agent transcripts side by side. Write 1–2 paragraphs each on:

- *Where* the failing arm went wrong — wrong plan, wrong tool call, hallucinated file path, gave up too early, looped?
- Whether the failure looks like a model capability issue or a harness mismatch (e.g., DeepSeek not respecting Claude Code's tool-use conventions).

This is the most valuable section of the writeup. The numbers tell you *whether* there's a difference; the transcripts tell you *why*. Lab announcements never include this.

8. Deliverables and repo structure

Your repo should look like this:

```
cc-deepseek-bench/
  README.md           # how to reproduce, in <1 hour
  ASSIGNMENT.md       # this document, copied in for reference
  pyproject.toml      # uv config
  tasks.txt           # the 20 task IDs you picked, one per line
  .env.example        # template; .env itself is gitignored
  .gitignore
  scripts/
    run-arm-a.sh      # the three sweep scripts
    run-arm-b.sh
    run-arm-c.sh
    aggregate.py       # parses Harbor JSON into a single CSV
  results/
    arm-a-anthropic/  # raw Harbor outputs (committed)
    arm-b-deepseek-pro/
    arm-c-deepseek-flash/
    combined.csv      # the aggregated long-format dataframe
  analysis.ipynb      # or analysis.md if you prefer
  FINDINGS.md         # 2-3 page writeup of conclusions
```

FINDINGS.md should answer, in this order:

1. Did the quality hypothesis hold? (with numbers and CIs)
2. Did the speed hypothesis hold? (with numbers)
3. What's the cost-per-resolved-task picture?
4. Where did the model swap *break* — i.e., tasks or behaviors specific to Claude that DeepSeek mishandled?
5. If you were advising a team deciding whether to use Claude Code with DeepSeek in production, what would you tell them?

Length: 2–3 pages of prose plus figures. Write it for a technical reader who hasn't read the methodology.

9. Reproducibility checklist

Before you call the assignment done, verify:

- Your README.md setup instructions, followed from scratch on a fresh machine, get someone to a passing hello-world run on all three arms in under an hour.
 - tasks.txt is committed and your scripts read from it (not hardcoded task lists).
 - All raw Harbor outputs are committed — someone else can re-run your analysis without re-running the benchmarks.
 - The exact Claude Code version, Harbor version, and Terminal-Bench dataset version are recorded in README.md. Pin them: `npm install -g @anthropic-ai/claude-code@<exact-version>`.
 - Your aggregate CSV has one row per (task, trial, arm) — long format — with columns for every metric in §6.4.
 - Per-task pricing math in your analysis is explicit and re-derivable. Don't hide it in a helper function with no comments.
-

10. Extra credit

Each of these is independent; pick whichever interests you.

10.1 More model backends. The same ANTHROPICBASEURL pattern works for several other providers — extend the study with any of these as additional arms:

- **z.ai (GLM 4.6)** — <https://api.z.ai/api/anthropic>, model glm-4.6. Chinese frontier model with strong coding benchmarks; an interesting third datapoint for the open-weight-models-are-catching-up thesis.
- **Moonshot / Kimi K2** — Anthropic-compatible endpoint; very long context, reasoning-heavy. Different shape than DeepSeek.
- **OpenRouter** — routes to almost anything (Grok, Llama, Qwen, Mistral, even xAI's models). Requires the y-router translation shim to bridge OpenAI format to Anthropic format. Document the setup.
- **A local model via Ollama or vLLM + a proxy** — the most technically interesting variant. Run a small model (Qwen 2.5 Coder 32B, or DeepSeek V3 GGUF) locally, expose it through an Anthropic-format adapter, and benchmark. You'll learn a lot about where the harness's expectations break.

For each new arm, report the same four-section analysis from §7 and add it to your final writeup. The interesting finding is rarely "which model wins" — it's usually *which task categories each model handles distinctively well or badly*, and what that tells you about model strengths beyond aggregate benchmark numbers.

10.2 SWE-bench Verified subset. Run the same three arms (Anthropic, DeepSeek V4-Pro, DeepSeek V4-Flash) against 15–25 instances of SWE-bench Verified using the official swebench evaluation harness. The setup is more involved — you'll need to either build a Claude-Code-on-SWE-bench adapter yourself, or use an existing one — but you'll be running on the most-cited industry benchmark for coding LMs, and you can directly compare your numbers to the published Claude and DeepSeek scores. The comparison is interesting because both providers report SWE-bench Verified results, but neither does so *through Claude Code*, so your numbers fill in a gap.

10.3 Cost-aware routing experiment. Build a simple router that uses DeepSeek V4-Flash for the first N turns of every task and only escalates to Anthropic if the agent looks stuck (e.g., repeated identical tool calls, no progress on tests). Measure whether you can get Anthropic-quality success rates at near-DeepSeek-Flash cost. This is the practical question every production team is going to ask once this kind of benchmark gets published.

11. Submission

When done, share the repo URL with me. I'll review for:

- Numbers I can re-derive from your committed data
- Methodology choices justified in writing
- An honest conclusion — falsifying the hypothesis is a perfectly good outcome if the data supports falsification
- Repo quality: someone else could pick this up and extend it

Questions before you start: ask. Questions after you've started running the sweep: write them down for the findings doc — they're often the most interesting part.

Good luck.